INFORME TÉCNICO

DESAFÍO II · Ude@Book

Universidad de Antioquia - Facultad de Ingeniería
Departamento de Ingeniería Electrónica y de Telecomunicaciones
Curso: Informática II (Semestre 2026-2)

Estudiante: David Salomon Ortiz Romero

Repositorio GitHub: https://github.com/dortiz02/Desafio-2-Info-2-2026-2

Video de sustentación (YouTube): https://youtu.be/84-46PNvU08

2026

1. Análisis del Problema y Modelo Funcional

Para este desafío desarrollamos Ude@Book, una red social académica en consola usando POO en C++17 puro. La aplicación debe soportar al menos 1200 usuarios con sus amistades bidireccionales, las solicitudes pendientes, los álbumes con distintos niveles de privacidad y el historial de mensajería privada de las últimas nueve semanas.

1.1. Restricciones técnicas

Por restricción del curso no usamos contenedores de la STL ni la palabra struct. No hay vector, list, map, set ni unordered_map; todas las colecciones dinámicas las manejamos con una plantilla propia Lista<T>. Tampoco hay herencia: las relaciones entre clases son composición, agregación o dependencia. Y está prohibido usar librerías de ANSI C (nada de printf, scanf, malloc, free, FILE*, strlen, strcpy ni las cabeceras <cstdio>, <cstring>, <cstdlib>); la E/S va con std::iostream y std::fstream, y la memoria con new / delete[].

Sobre std::string, solo usamos operaciones básicas como .size(), operator[], .capacity() y .data(). Ningún algoritmo llama a find, substr, insert, erase ni replace. Por eso la comparación, validación, hash y partición de enteros las hicimos carácter por carácter en la clase utilitaria Cadena.

2. Diseño Orientado a Objetos

2.1. Diagramas UML

(Aquí van las imágenes exportadas de diagrama_clases_resumen.drawio y diagrama_clases_uml.drawio que están en la raíz del proyecto.)

2.2. Responsabilidad de cada clase

El sistema se divide en 12 clases y una enumeración fuertemente tipada (enum class Privacidad):

1. Lista<T> (include/lista.h). Es una plantilla de contenedor lineal dinámico genérico sobre memoria contigua (T* datos). Crece multiplicando por 2, inserta al final en O(1) amortizado, permite inserción ordenada (insertarOrdenado) y búsqueda binaria (buscarBinaria) en O(log n). Cumple la Regla de los Tres con constructor de copia, operador de asignación y destructor.
2. Cadena (include/cadena.h y src/cadena.cpp). Agrupa utilidades de cadena carácter por carácter: comparación exacta (iguales), detección de cadena vacía (esVacio), dispersión DJB2 (hash), conversión y separación de enteros por delimitador (dividirEnteros, aEntero) y cálculo de memoria dinámica (bytesDinamicos).
3. Fecha (include/fecha.h y src/fecha.cpp). Maneja fechas y horas con formato dd/mm/aaaa hh:mm, valida años bisiestos y días por mes. Para comparar fechas rápido genera una clave entera (claveDia = anio * 10000 + mes * 100 + dia), de modo que las comparaciones quedan en O(1). También sobrecarga los operadores relacionales (==, !=, <, <=, >, >=) y operator<< para impresión.
4. Foto (include/foto.h y src/foto.cpp). Representa una foto dentro de un álbum, con fecha de captura (Fecha), tamaño en KB, ruta de imagen y un contador de likes que se modifica con darLike y quitarLike. Sobrecarga operator== y operator<<.
5. Album (include/album.h y src/album.cpp). Guarda una Lista<Foto>, el nombre, la ruta en disco y el nivel de privacidad (Privacidad::Publico, Privacidad::Amigos o Privacidad::Privado). Con esVisibleParaAmigos() decide si el álbum puede verse desde el perfil de un amigo.

6. Usuario (include/usuario.h y src/usuario.cpp). Es la entidad principal. Encapsula los datos del perfil (nombre único, fecha de nacimiento, correo, ruta de foto de perfil, clave, ciudad, país y fecha de inscripción), sus álbumes (Lista<Album*>) y cinco listas de ids enteros (Lista<int>):

- amigos: siempre ordenada de menor a mayor, para buscar en O(log g) e intersectar en O(g_a + g_b).
- solicitudesRecibidas y solicitudesEnviadas.
- mensajesRecibidos y mensajesEnviados: ordenadas por fecha de envío.

7. Mensaje (include/mensaje.h y src/mensaje.cpp). Representa un mensaje privado entre dos usuarios (idEmisor, idReceptor, Fecha, estado leido y texto de hasta 512 caracteres). Sobrecarga operator< (compara por fecha) y operator<<.

8. IndiceUsuarios (include/indiceusuarios.h y src/indiceusuarios.cpp). Implementa una tabla hash de direccionamiento abierto con sondeo lineal y factor de carga menor a 0.5 (capacidad = 2 * N + 1). Asocia cada nombre con su id entero en O(1) promedio, sin recorrer los 1200 usuarios.

9. RedSocial (include/redsocial.h y src/redsocial.cpp). Es la clase controladora del dominio. Por composición es dueña de todos los Usuario*, de todos los Mensaje* y del IndiceUsuarios. Implementa la autenticación, la selección aleatoria ponderada de los 3 mejores amigos, el envío y aceptación de solicitudes (enviarSolicitud está sobrecargada por ID y por nombre), la sugerencia de k amigos por afinidad de nivel 2 y la búsqueda binaria de mensajes por rango de fechas.

10. Almacenamiento (include/almacenamiento.h y src/almacenamiento.cpp). Se encarga de la persistencia en data/ude_datos.dat con serialización binaria directa usando std::ifstream y std::ofstream en modo std::ios::binary.

11. Medidor (include/medidor.h y src/medidor.cpp). Lleva las métricas de eficiencia que pide la guía en cada funcionalidad: iteraciones de ciclos directos, iteraciones dentro de subfunciones llamadas, invocaciones a librerías externas (std::uniform_real_distribution) y memoria consumida, tanto en estructuras como en variables locales de la pila.

12. Consola (include/consola.h y src/consola.cpp). Es la interfaz por consola. Valida entradas y arma los menús tal como pide la guía, incluyendo la opción 'N' para alternar una sola vez por sesión las 3 fotos de amigos en el perfil y la opción 'S' para cerrar sesión.

3. Estructuras de Datos y Formato Binario

3.1. Por qué usamos ids enteros en memoria

En lugar de duplicar objetos o cruzar punteros entre usuarios, cada Usuario y cada Mensaje recibe un id entero secuencial (0 .. N-1 y 0 .. M-1) que corresponde a su posición en RedSocial::usuarios y RedSocial::mensajes.

En memoria, esto tiene una ventaja clara: cada amistad, solicitud o referencia a mensaje ocupa solo 4 bytes (sizeof(int)), en lugar de un Usuario o un Mensaje completo.

En algoritmos también ayuda. La lista amigos se arma con Lista<int>::insertarOrdenado, así que podemos saber si dos usuarios son amigos con búsqueda binaria en O(log g) y calcular amigos en común con un recorrido simultáneo de dos índices (merge-intersection) en O(g_a + g_b), que es mucho mejor que O(g_a * g_b). Además, como mensajesRecibidos queda ordenada por fecha, filtrar mensajes entre dos fechas se hace con dos búsquedas binarias (posicionLimite) en O(log m + r), sin recorrer todos los mensajes del usuario.

3.2. Diseño del archivo binario (data/ude_datos.dat)

Armamos un formato binario propio con encabezado mágico (UDEB, versión 1), documentado en data/formato.txt. La razón de usar binario y no texto plano es el costo: al cargar 1200 usuarios, 17,313 amistades y 15,000 mensajes, parsear delimitadores tomaría tiempo. Con binario la carga completa e indexación en memoria tarda pocos milisegundos.

Bloque 1 (Encabezado): char[4] con "UDEB", int32 version = 1, Fecha fechaActual (5 enteros de 32 bits), int32 totalUsuarios, int32 totalAmistades, int32 totalSolicitudes y int32 totalMensajes.

Bloque 2 (Usuarios, totalUsuarios): cadenas con formato int32 longitud + bytes para nombre, correo, rutaFotoPerfil, clave, ciudad y pais; Fecha fechaNacimiento y Fecha fechaInscripcion; int32 cantAlbumes y luego cada Album (nombre, ruta, int32 privacidad, int32 cantFotos, y cada Foto con Fecha, tamanoKB, ruta, likes).

Bloque 3 (Amistades, totalAmistades): pares (int32 idUsuarioA, int32 idUsuarioB) con idUsuarioA < idUsuarioB. Al cargar, las insertamos en ambas direcciones.

Bloque 4 (Solicitudes, totalSolicitudes): pares (int32 idEmisor, int32 idReceptor) de solicitudes pendientes.

Bloque 5 (Mensajes, totalMensajes): int32 idEmisor, int32 idReceptor, Fecha fechaEnvio, uint8 leido y cadena texto.

Todo se escribe en little-endian.

4. Algoritmos Esenciales

4.1. Indexación y búsqueda de usuarios (IndiceUsuarios::buscar). Calculamos el hash DJB2 del nombre en O(L), donde L es la longitud del nombre, y hacemos sondeo lineal sobre un arreglo de tamaño 2N + 1. En promedio la complejidad es O(1).

4.2. Selección aleatoria ponderada de los 3 mejores amigos (RedSocial::elegirMejoresAmigos). Para cada amigo i del usuario activo calculamos amigosEnComun(idUsuario, amigos[i]) con intersección ordenada en O(g_u + g_i). El peso queda en 1.0 si es un amigo normal, 2.5 si tienen más de 10 amigos en común y 3.5 si tienen más de 20. Si el usuario presiona 'N' para alternar, los 3 amigos mostrados al inicio reciben peso 0.0, y solo se restauran si tiene menos de 6 amigos. Después hacemos 3 extracciones aleatorias ponderadas sin reemplazo con std::mt19937 y std::uniform_real_distribution<double>.

4.3. Sugerencia de k amigos por afinidad (RedSocial::sugerirAmigos). Creamos contador[totalUsuarios] en 0 y marcamos con -1 al propio usuario, a sus amigos actuales y a los que tienen solicitudes pendientes. Luego recorremos las listas de amigos de cada amigo del usuario (vecinos a distancia 2) y vamos incrementando contador[candidato]. Al final, los candidatos con contador > 0 se insertan ordenadamente en un arreglo de tamaño k, de mayor a menor cantidad de amigos en común.

4.4. Búsqueda de mensajes por rango de fechas (RedSocial::buscarPorRango). Convertimos las fechas inicial y final a su clave entera AAAAMMDD con Fecha::claveDia(), y llamamos dos veces a la búsqueda binaria posicionLimite sobre mensajesRecibidos: una con estricto = false (lower bound del día inicial) y otra con estricto = true (upper bound del día final). Extraemos el subrango [desde, hasta).

5. Eficiencia y Uso de Recursos

Con la base cargada (1,200 usuarios, 17,313 amistades, 400 solicitudes pendientes y 15,000 mensajes repartidos en las últimas 9 semanas), la memoria total de las estructuras del sistema (RedSocial::memoriaTotal()) es de 3,108,712 bytes, es decir aproximadamente 2.96 MB.

Para medir, usamos la clase Medidor y ejecutamos cada funcionalidad con el usuario de prueba ana0. Los resultados se resumen en la tabla de abajo.

Funcionalidad	Iter. Directas	Iter. Indirectas	Llamadas Ext.	Mem. Estructuras (bytes)	Mem. Pila (bytes)	Complejidad
Inicio de sesión + selección de 3 mejores amigos (ana0)	291	18,452	3	3,108,712	763	O(g * g_prom)
Alternar 3 mejores amigos en el perfil ('N')	291	18,430	3	3,108,712	763	O(g * g_prom)

Enviar solicitud (enviarSolicitud)	0	24	0	3,108,712	20	O(log g)
Aceptar solicitud (aceptarSolicitud)	0	31	0	3,108,712	20	O(g)
Visitar perfil de amigo y recorrer álbumes	2	12	0	3,108,712	16	O(1) por foto
Sugerir k = 5 amigos (sugerirAmigos)	2,148	410	0	3,108,712	6,956	O(N + g * g_prom + C * k)
Enviar mensaje privado (enviarMensaje)	0	48	0	3,108,856	40	O(L)
Leer mensajes nuevos no leídos (recopilarNoLeidos)	18	6	0	3,108,856	24	O(m_rec)
Buscar mensajes por rango de fechas (buscarPorRango)	12	10	0	3,108,856	28	O(log m_rec + r)

Dos observaciones sobre las métricas:

- Con IndiceUsuarios, buscar un usuario por nombre gasta entre 6 y 15 iteraciones indirectas en promedio, que corresponden a los caracteres del nombre en Cadena::hash y Cadena::iguales. Una búsqueda secuencial sobre 1200 usuarios gastaría cerca de 600 comparaciones promedio.
- La búsqueda por rango de fechas hace alrededor de 10 iteraciones indirectas de búsqueda binaria, sin importar cuántos mensajes haya en total en la red (15,000 en este caso).

6. Problemas Encontrados, Alternativas y Escalabilidad

6.1. Problemas y cómo los resolvimos

Lo primero que nos dio problemas fue la prohibición de find, substr y los contenedores de la STL. Para validar las entradas de consola (fechas dd/mm/aaaa y enteros) sin usar find, substr ni stringstream, terminamos implementando el autómata carácter por carácter Cadena::dividirEnteros, que valida dígitos, detecta desbordamientos y separa campos en una sola pasada lineal.

Un caso borde que nos costó fue el de alternar las 3 fotos de amigos con 'N'. Si un usuario tiene 3 amigos o menos, o entre 4 y 5, al excluir los 3 iniciales quedaban menos de 3 candidatos con peso positivo y el programa no completaba las 3 fotos. Lo solucionamos detectando cuándo la suma de pesos restantes cae a 0.0 (total <= 0.0) y, en ese caso, restaurando los pesos base de los amigos aún no elegidos en la tanda actual. Así siempre se completan 3 amigos sin repetir ninguno dentro del mismo grupo.

También tuvimos problemas con las rutas al compilar con Qt Creator en Shadow Build. Como el proyecto se compila en un directorio separado (build-...), la ruta relativa hacia data/ude_datos.dat cambia según la profundidad del directorio de compilación. Al principio las pruebas fallaban por eso. Lo resolvimos con buscarArchivoDatos() en src/main.cpp, que revisa automáticamente rutas relativas ascendentes (data/ude_datos.dat, ../data/ude_datos.dat, etc.) sin depender de rutas absolutas fijas.

6.2. Qué cambiaría si la red social crece a millones de usuarios

Si Ude@Book pasara de 1.2 * 10^3 usuarios a 10^7 o 10^9 activos, no podríamos seguir cargando todo en RAM de un solo proceso.

Lo primero que habría que cambiar es el almacenamiento. No tendría sentido tener todos los usuarios, álbumes y mensajes en memoria al mismo tiempo (RedSocial::usuarios). En vez de eso, habría que dividir el archivo .dat en bloques y guardar un índice de posiciones (offsets) en disco. Cuando un usuario inicie sesión, se haría seekg a la posición correspondiente y se leerían solo sus datos, dejando el resto en disco. Para los mensajes habría que hacer lo mismo, con un archivo aparte por usuario o por bloque.

La función sugerirAmigos también tendría que cambiar. Hoy usamos un arreglo temporal int* contador = new int[totalUsuarios], que para N = 1200 ocupa unos 4.8 KB y funciona bien. Pero si N fuera 10^8, ese arreglo pediría 400 MB por cada consulta, lo cual no es viable. Tocaría reemplazarlo por una estructura más chica que solo guarde los ids de los usuarios que sí tienen al menos un amigo en común, por ejemplo una lista de candidatos que se va llenando a medida que se recorre.

Finalmente, elegirMejoresAmigos hoy recalcula amigosEnComun en cada inicio de sesión. Con millones de usuarios eso sería muy costoso. Una mejora razonable sería mantener precalculado el número de amigos en común de cada arista (u, v) y actualizarlo de forma incremental cada vez que se acepta una amistad nueva, así el cálculo al iniciar sesión sería casi inmediato.

7. Conclusiones

1. Dividir el sistema en 12 clases desacopladas nos permitió cumplir todo lo funcional y lo técnico del desafío sin STL, sin struct, sin herencia y sin funciones de ANSI C.
2. Combinar una tabla hash con sondeo lineal (IndiceUsuarios) con arreglos dinámicos ordenados y búsqueda binaria (Lista<int>) bajó mucho el número de iteraciones en autenticación, en la intersección de amigos en común y en el filtrado de mensajes por fecha.
3. El formato binario propio (ude_datos.dat) conserva el estado entre ejecuciones (solicitudes nuevas, amistades, likes, fotos de perfil y mensajes leídos o enviados) con muy poca sobrecarga de lectura y escritura.